25901194

MAHATMA GANDHI UNIVERSITY, KOTTAYAM

MGU-UGP (HONOURS) REGULAR EXAMINATION March 2026

Fourth SEMESTER

Discipline Specific Elective (DSE) – MG4DSECSC202 – SYSTEM PROGRAMMING

(2024 ADMISSION ONWARDS)

Duration: 1 Hour 30 minutes **Maximum Marks: 50**

Remember(K), Understand(U), Apply(A), Analyse(An), Evaluate(E), Create(C), Skill(S), Interest(I) and Appreciation(Ap)

Part A

Very Short Answer Questions

Answer all questions

Each question carries **2** marks

1. Identify the role of the symbol table in Pass-1 **[K] / [CO1]**

In Pass-1 of a two-pass assembler, the symbol table's primary role is to construct a mapping of labels (user-defined symbols) to their corresponding memory addresses, tracking their locations and attributes. It acts as a central repository for defining symbols, allowing the assembler to resolve future references.

2. Provide an example assembly instruction with all fields labeled **[K] / [CO1]**

An assembly instruction consists of four main fields: label, mnemonic (instruction), operands (data), and comment.

Eg: loop_start: MOV AX, 5 ; Initialize loop counter

3. **Define a macro**. **[K] / [CO2]**

A macro is a symbolic name, rule, or pattern that represents a sequence of instructions, code, or actions. It acts as a shortcut that the compiler or software replaces with a pre-defined, more complex sequence to automate repetitive tasks, improve readability, or enable code reuse.

4. **Interpret the compilation of control structures.** **[U] / [CO2]**

Compilation of control structures translates high-level logic—such as `if-else` (selection), `for` / `while` (iteration), and `switch` (branching)—into low-level machine code or intermediate representation (IR) using jumps, labels, and comparisons. This process converts structured programming concepts into optimized conditional and unconditional branching to manage program execution flow.

Key Components of Compiled Control Structures

Selection Structure (If-Else): Compiled into a conditional branch, where the condition is evaluated, and the instruction pointer jumps over the non-matching block. The {Link: Study.com https://study.com/academy/lesson/python-control-structures.html} provides details on Python control structures.

Iteration Structure (Loops): Compiled to evaluate a condition, execute a body, and loop back (a jump) to the condition check until it is false, says {Link: Study.com https://study.com/academy/lesson/control-structures-definition-types-examples.html} .

Switch Statement: Often compiled into jump tables (for dense cases) or a series of conditional jumps (for sparse cases) for efficient multi-branching, as seen on {Link: Duke University https://fintechpython.pages.oit.duke.edu/jupyternotebooks/3-CPlusCPlus/04-ControlStructures/04-ControlStructures.html}

5. **Define relocation table.** **[K] / [CO3]**

A relocation table is a data structure, typically found in object files or executable files, that

lists all the memory addresses in a program's code and data sections that need to be updated (patched) by the linker or loader. It allows a program to execute correctly at any memory address, supporting dynamic linking and ASLR (Address Space Layout Randomization).

Part B

Short Answer Questions

Answer any **5** questions

Each question carries **5** marks

6. **Compare one-pass and two-pass assembly schemes.** **[U] / [CO1]**

One-pass assemblers scan source code once, offering faster assembly by resolving symbols immediately or using back-patching, but struggle with forward references. Two-pass assemblers scan twice—first to build the symbol table, then to generate code—allowing efficient handling of forward references, complex expressions, and better error detection.

Key Differences:

Passes over Source Code: One-pass scans once; Two-pass scans twice (first pass for definitions, second for translation).

Forward References: One-pass has difficulty handling symbols used before they are defined (forward references), often requiring "back-patching". Two-pass handles forward references efficiently by defining all symbols in the first pass.

Speed: One-pass is generally faster as it requires less I/O and processing.

Symbol Table: In two-pass, the symbol table is complete before code generation, whereas in one-pass it is built on-the-fly.

Complexity: One-pass is simpler to design but limited; Two-pass is more complex but more powerful.

Use Case: One-pass is suitable for simple, straightforward code or small systems, while two-pass is used for complex programs.

Feature	**One-Pass Assembler**	**Two-Pass Assembler**

Passes	Scans code once	Scans code twice
Forward References	Cannot easily resolve	Handles seamlessly
Speed	Faster	Slower
Symbol Table	Built during translation	Built in Pass 1, used in Pass 2
Back-patching	Usually required	Not required
Complexity	Simple	Complex
Error Detection	Limited	Better

One-Pass Assembler: The primary challenge is that when a label is used as an operand (e.g., `JMP START`), the address of `START` may not be known yet. It handles this by leaving the address blank, recording the location of the operand, and "back-patching" (filling in the address) once the symbol is finally defined.

Two-Pass Assembler: The first pass defines all labels and their addresses to build a complete symbol table. The second pass then translates instructions into machine code using the complete table, enabling complex addressing.

7. Explain the parsing process using a simple grammar and diagram. [U] / [CO1]

Parsing is the process of analyzing a string of tokens (from a program or sentence) against grammar rules to create a defining tree. It determines if an input is valid according to a language's syntax and sets up the foundation for interpretation or execution.

1. Simple Grammar example

We will use this small Context-Free Grammar (CFG) to define a simple assignment:

Production Rules:

1. S →id=E(Statement)

2.E→id+id (Expression)

Input String: `x = y + z`

Tokens: `id`, `=`, `id`, `+`, `id`

2. The Parsing Process (Top-Down Example)

Lexical Analysis (Scanner): Converts `x = y + z` into `id = id + id`.

Syntax Analysis (Parser): Takes the tokens and tries to build a parse tree from top to bottom (start symbol S).

Applying Rules:

Match X→id=E (Matches `id` to `x`, `=` to `=`, leaves `E` to match `y + z`).

Match E→id+id (Matches `id` to `y`, `+` to `+`, `id` to `z`).

Completion: All tokens are matched, and a complete parse tree is formed.

3. Parse Tree Diagram (`x = y + z`)

```
text
    S
  / | \
 id  =  E


 |   / | \
 x  id  +  id


   |     |
   y     z
```

Root: Start Symbol **(S).**

Internal Nodes: Non-terminals **(S,E).**

Leaf Nodes: Terminals (`id, =, +`).

Sentence: Reading the leaves left-to-right (x = y + z) confirms the input.

8. **List the functions of system software and explain any three.** **[K] / [CO1]**

System software manages computer hardware, provides a platform for application software, and ensures system efficiency. Key functions include memory management, process management, file management, input/output control, device management, and security. It acts as a bridge between users and hardware.

Functions of System Software

Memory Management: Allocates and deallocates memory space for programs.

Process Management: Manages CPU scheduling to ensure applications run smoothly.

File Management: Organizes, stores, and manages files and directories on storage devices.

Input/Output Control: Manages communication between the CPU and peripherals (keyboard, monitor, printer).

System Security: Protects data via firewalls and user access controls.

Device Management: Utilizes drivers to interface with hardware components

Memory Management:

The system software (specifically the operating system) keeps track of every memory location, whether it is allocated to a program or free. It allocates memory when a process requests it and deallocates it when the process completes, ensuring no application interferes with another's memory space.

Process Management:

This function handles the execution of applications. Since a CPU can generally do one thing at a time, the system software determines which process gets the processor, for how long, and in what order (scheduling). It manages process creation, termination, and

synchronization.

Input/Output(I/O)Management:

The system software provides a consistent interface to manage diverse hardware devices. It manages input from devices like keyboards and mice, and controls output to monitors or printers, ensuring data is sent and received correctly without hindering the main system performance.

9. **Illustrate the use of debugging tools in detecting logical and runtime errors.** **[A] / [CO2]**

Debugging tools, often integrated into IDEs like Visual Studio or standalone tools like GDB (GNU Debugger), allow developers to control program execution and inspect its state to identify and resolve both logical and runtime errors.

GeeksforGeeks +1

Detecting Logical Errors

Logical errors occur when a program runs without crashing but produces incorrect output because of a flaw in its logic. Debugging tools help by revealing the program's actual behavior versus the expected behavior.

YouTube +3

Example: Incorrect Temperature Conversion

A program intends to convert Celsius to Kelvin with the formula: `Kelvin = Celsius + 273.15`, but due to a coding error, performs string concatenation.

Problem Code (JavaScript):

```
let degreesC = input.question('Temp in degrees C: '); // User enters 100

let degreesK = degreesC + 273.15;

console.log('Degrees K:', degreesK); // Output: Degrees K: 100273.15 (Incorrect, should be 373.15)
```

Debugging Process using Tools:

Set a Breakpoint: A developer places a breakpoint on the line `let degreesK = degreesC + 273.15;` to pause execution just before the calculation.

Inspect Variables: When the program pauses, the developer uses the debugger's variable watch window to inspect the `degreesC` variable. They find its value is `'100'` (a **string**)

instead of `100` (a **number**).

Identify the Cause: The debugger reveals that the `+` operator performs string concatenation because one operand is a string, which is the root cause of the incorrect output.

Fix and Verify: The developer modifies the code to explicitly convert the input to a number (`degreesC = Number(degreesC);`) and re-runs the program in debug mode to confirm the correct output (`373.15`)

Detecting Runtime Errors

Runtime errors cause a program to crash or stop during execution, often due to an impossible or disallowed operation. Debugging tools pinpoint the exact line of failure and the program state at that moment.

YouTube +3

Example: Division by Zero

A program attempts to perform a mathematical operation that is impossible during execution.

Problem Code (Python):

```
numerator = 10
denominator = 0
result = numerator / denominator # Runtime error occurs here
print(result)
```

Debugging Process using Tools:

Error Message: The program crashes and generates an error message like `ZeroDivisionError: division by zero`. The message often includes the specific file and line number where the error occurred.

Analyze the Crash: The developer uses the debugger to load a crash dump or rerun the program in debug mode. The debugger automatically stops at the line causing the error.

Examine the Call Stack and Variables: The debugger's call stack feature shows the sequence of function calls leading to the error, and the variables window shows `denominator` has a value of `0`.

Identify the Cause: The developer sees that the logic allows the denominator to become zero, an invalid operation.

Fix and Verify: The developer adds an assertion or conditional check to prevent division by zero, then runs tests to validate the fix.

10. Differentiate between interpreters and compilers based on execution and memory usage. **[U] / [CO2]**

Compilers and interpreters differ primarily in how and when they translate source code into machine code, which impacts execution speed and memory usage.

Execution Differences

Compiler: A compiler translates the entire source code into a standalone machine code (executable) file before the program runs. This pre-translation process allows for significant code optimization, resulting in much faster program execution once the executable is generated.

Interpreter: An interpreter translates and executes the source code line by line during runtime. It doesn't produce an independent executable file; instead, the interpreter program reads and processes the code on the fly with each execution. This results in slower overall execution compared to compiled code because the translation overhead occurs every time the program runs.

Memory Usage Differences

Compiler: During the initial compilation process, a compiler requires more memory to analyze the entire codebase and generate intermediate object code. However, the resulting standalone executable file generally uses less memory during runtime because the source code and the compiler itself are not needed for execution.

Interpreter: An interpreter uses more memory during execution because the interpreter software itself must be loaded into memory along with the source code (or an intermediate form like bytecode) to perform real-time translation. It does not generate the large, persistent object files that compilers do, making it more memory efficient in terms of storage space, but more demanding on runtime memory.

Feature	Compiler	Interpreter
Translation	Translates the entire program at once before execution.	Translates and executes code li line during runtime.

Execution Speed	Faster execution of the resulting program.	Slower execution due to real-time translation.
Memory at Runtime	Uses less memory during program execution.	Uses more memory during program execution as the interpreter must run simultaneously.
Intermediate Code	Generates an intermediate object code and a standalone executable file.	Does not generate a separate executable file.
Debugging	Errors are reported all at once after compilation, making debugging more challenging.	Stops at the first error, making debugging easier and more interactive.
Use Cases	Suitable for large, performance-critical applications (e.g., C, C++, C#).	Ideal for scripting, rapid development, and testing (e.g., Python, JavaScript, Ruby).

11. Describe the working of a relocating loader. **[U] / [CO3]**

A relocating loader is a system software component that loads programs into arbitrary memory locations, enabling efficient memory usage by allowing multiple programs to share memory. It works by taking the object code, assigning it to a new base address, and modifying all address-dependent instructions by adding an offset.

How a Relocating Loader Works:

Memory Allocation: Upon receiving an object program, the loader asks the operating system for a free space to hold the program, determining the starting address for loading, often referred to as the "load address" or "base address".

Relocation Identification: The loader reads the object code to find address-sensitive instructions (like branching/JMP, data references, or memory addresses) that were originally assembled with a base address of zero.

Modification Techniques:

Relocation Bits: The object code has bits where '1' means the instruction requires

modification (relocation), and '0' means it does not.

Modification Records: The object code includes records that specify exactly which locations need to be modified based on the starting address.

Loading and Linking: The loader moves the modified code into the designated main memory locations and resolves symbolic external references if multiple subroutines are combined.

Execution Transfer: Finally, it transfers control to the starting address of the loaded program for execution.

Key Advantages:

Flexible Memory Usage: Programs do not need to be loaded at a pre-set memory address.

Support for Multi-programming: Enables efficient sharing of memory among different programs by allowing them to load into whatever space is currently available

12. **Explain the role of overlay manager.** **[U] / [CO3]**

An overlay manager acts as a central coordinator, optimizing performance either by managing memory in computer systems or by overseeing investment portfolios. In technology, it loads program parts into limited memory, while in finance, it manages risk and asset allocation across multiple portfolios using derivatives.

Key Roles in Investment/Finance:

Unified Strategy: The overlay manager maintains a consistent strategy across multiple managers, preventing conflicting trades and optimizing overall portfolio allocation.

Risk Management & Hedging: They use derivative instruments (futures, options, swaps) to hedge against market downturns, manage currency risks, and manage downside tail events.

Cash Equitization: They convert cash holdings into market exposure to prevent "cash drag," ensuring that cash earns market returns.

Efficiency: They rebalance portfolios efficiently, often reducing transaction costs by limiting the need for buying/selling underlying securities.

Key Roles in Computer Science (Memory Management):

Memory Optimization: In systems with limited memory, the overlay manager loads only necessary program segments (overlays) into memory, allowing larger programs to run.

Dynamic Loading: It handles loading and unloading of these segments from disk to RAM as they are needed.

Key Roles in Network Infrastructure:

Overlay Networks: They manage virtualized networks (like VPNs or SD-WAN) operating on top of physical infrastructure, directing traffic and improving security without altering the underlying network

Part C

Essay Questions

Answer any **1** questions

Each question carries **15** marks

13. **Describe the design and working of a macro preprocessor with neat diagrams.** **[U] / [CO2]**

A macro preprocessor is a system software that expands macro instructions (shorthand codes) into corresponding source language statements *before* compilation or assembly. It operates by substituting macro calls with their stored body definitions, aiding readability and reducing code repetition without analyzing the meaning of the statements.

1. Structure and Design of a Macro Preprocessor

The design of a macro preprocessor focuses on identifying macro definitions and replacing macro calls with expanded code. It generally consists of two passes (or can be done in one) using several key data structures:

Macro Definition Table (MDT): Stores the body of the macro definition (the lines of code).

Macro Name Table (MNT): Stores the names of defined macros and pointers to their location in the MDT.

Argument List Array (ALA) / Argument Table (ARGTAB): Stores the formal parameters during definition and actual arguments during expansion.

Working Mechanism

The macro processor functions in two main phases:

Macro Definition (Pass 1):

The preprocessor reads the source program.

When it encounters a `MACRO` pseudo-op, it saves the macro name in the MNT and the body in the MDT.

It replaces formal parameters (e.g., `&ARG1`) with positional notation (e.g., `#1`).

It continues until it hits `MEND` (Macro End).

Macro Expansion (Pass 2):

The preprocessor reads the source again.

When an operation code matches an entry in the MNT, it's identified as a macro call.

It replaces the call with the body retrieved from the MDT, substituting formal parameters with actual arguments from the source code

Example of Macro Preprocessor Operation

Input Code:

```
MACRO_SUM MACRO &A, &B
    LOAD &A
    ADD &B
MEND

START:
    MACRO_SUM X, Y
    MACRO_SUM P, Q
```

After Processing (Output):

```
START:
    LOAD X
    ADD Y
```

LOAD P

ADD Q

Key Data Structure Interaction

MNT: Contains `[Name: MACRO_SUM, MDT_Index: 1]`.

MDT: Contains `[1: LOAD #1, ADD #2]` (where #1 and #2 are parameters).

ALA: Stores `[1: X, 2: Y]` for the first call.

The preprocessor uses the ALA to replace `#1` with `X` and `#2` with `Y` during expansion.

14. Analyze the concepts of linking and relocation with suitable examples and diagrams. [An] / [CO3]

Linking and Relocation are essential steps in the transformation of source code into an executable program. **Linking** combines separate object modules and libraries into a single executable, resolving external references. **Relocation** modifies the address-dependent locations in the object program so it can be loaded into, and executed from, any memory location.

1. Linking (Linkage Editing)

Linking is the process of collecting and merging multiple object modules (compiled `.obj` files) and library routines into a single executable file (e.g., `.exe` or `.out`). A program called a **linker** (or linkage editor) performs this task.

Key Aspects of Linking:

- **Symbol Resolution:** The linker resolves references between object files. If `main.c` calls `func()`, the linker matches that call to the definition of `func()` in `another.c`.
- **Library Search:** Linkers search for library functions (like `printf`) to include in the executable.

<u>Types of Linking</u>:

- **Static Linking:** Necessary library code is copied into the final executable at compile time. It is self-contained but creates larger files.

- **Dynamic Linking:** Libraries are not included in the executable. Instead, they are loaded at runtime (e.g., `.dll` files in Windows, `.so` in Linux). This allows multiple programs to share one library copy, reducing memory usage.

Example of Linking:

```
// main.c - Source Module A
extern int add(int a, int b);
int main() { return add(2, 3); }

// lib.c - Source Module B
int add(int a, int b) { return a + b; }
```

Relocation

Relocation is the process of adjusting the address-dependent portions of a program so it can be loaded into any memory area. It allows programs to run regardless of the physical memory location allocated.

WhyRelocation?

In modern systems, programs are not loaded at a fixed address because the operating system needs to load multiple programs at different times, often leading to fragmented memory. The loader must update addresses within the program to match its new, allocated memory space.

Types of Relocation:

- **Static Relocation:** The loader fixes all addresses at load time. Once loaded, the program cannot be moved.
- **Dynamic Relocation:** The processor translates addresses at runtime using a **Relocation Register (or Base Register)**. The program thinks it starts at `0x0000`, but the actual address is `0x0000 + Relocation Register`.

ExampleofRelocation:

If an instruction says `JUMP 1000`, and the program is loaded starting at `5000` instead of `0`, the address must be adjusted to `JUMP 6000` (5000 + 1000).

3. Conceptual Diagrams

The overall process involves transforming source code into a runnable program image.

A. Linking and Loading Process

This diagram illustrates how source codes are translated, linked, and eventually loaded.

B. Relocation Concept (Address Translation)

This diagram shows how a logical (relative) address is mapped to a physical (absolute) address using a relocation register.

Summary Table: Linking vs. Relocation

Feature	Linking	Relocation
Purpose	Combine multiple modules/libraries.	Adjust addresses to load in memory.
Input	Multiple Object Files (`.o`).	Linked Executable (`.exe`).
Output	Single Executable File.	Relocated/Loaded Memory Image.
Resolves	External symbols (`EXTRN` / `PUBLIC`).	Address-dependent instructions.
Time	Link-time (before loading).	Load-time or Run-time.